



TÉCNICO SUPERIOR EN DESARROLLO DE APLICACIONES WEB

DEPARTAMENTO DE INFORMÁTICA



MANUAL TÉCNICO

Autor/es: María Ceballos Mesias

Curso Académico: 2025/26

ÍNDICE

CONTENIDO

1.Introducción	3
1.1 descripción del proyecto	3
1.2 Justificación y Problemática.....	3
1.3 Ámbito del Proyecto.....	4
1.4 Diagrama de Contexto	5
2. Arquitectura de la aplicación	6
2.1 fronted.....	6
2.1.1 Tecnologías usadas	6
2.1.2 Entornos de desarrollo	7
2.2 Backend.....	8
2.2.1 Tecnologías usadas	8
2.2.2 Entorno de desarrollo.....	8
3. Documentación técnica.....	9
3.1 ANÁLISIS	9
3.1.1 REQUISITOS FUNCIONALES	9
3.1.2 REQUISITOS NO FUNCIONALES	9
3.2 DesARROLLOS.....	10
3.2.1) DISEÑO de la BBDD	10
3.2.2) Modelo Entidad/Relación de la BBDD.....	11
3.2.3 Modelo relacional de la BBDD	12
3.2.4 DIAGRAMA DE CASOS DE USO	20
3.3 PRUEBAS REALIZADAS.....	21
3.3.1 Pruebas Funcionales	21
3.3.2 Pruebas de seguridad (rls)	23
3.3.3 Pruebas de Interfaz y Responsividad.....	23
3.4 Implementación del Agente IA (SaniclearBot)	23
3.4.1 ARQUITECTURA DEL AGENTE	23
3.4.2 Especificaciones Técnicas y Lógica de Inyección	24
3.4.3 Funcionalidades del Asistente	24
4. Proceso de despliegue	25
4.1 ADQUISICIÓN DE DOMINIO Y CORREO CORPORATIVO EN CLOUDFLARE.....	25
4.2 DESPLIEGUE DEL BACKEND.....	26

Proyecto “Desarrollo de Aplicaciones Web”

Título del Proyecto: SANICLEARS



JUNTA DE EXTREMADURA

Consejería de Educación y Empleo

4.3	DesPLIEGUE DEL FRONTED	27
4.4	REQUISITOS DEL SISTEMA.....	29
5.	MANUAL DE OPERACIÓN	30
5.1	ROLES Y PERMISOS	30
5.2	Módulo Superadmin — Descripción Detallada.....	30
5.3	SaniclearsBot — Asistente de IA	31
5.4	FLUJO DE TRABAJO TÍPICO	32
6.	PROPUESTA DE MEJORAS	32
6.1	MEJORAS APLICADAS DURANTE EL DESARROLLO.....	32
6.2	MEJORAS PROPUESTAS PARA FUTURAS VERSIONES	33
6.3	REQUISITOS TÉCNICOS.....	34
6.3.1	BACKEND	34
6.3.2	FRONTED	34
7.	CREDENCIALES DE ACCESO.....	34
8.	BIBLIOGRAFÍA	35
8.1	WEBS DE REFERENCIA.....	35
8.2	ARTÍCULOS DE REFERENCIA.....	35

1.INTRODUCCIÓN

1.1 DESCRIPCIÓN DEL PROYECTO

El proyecto denominado **SANICLEARS** consiste en el diseño, desarrollo e implementación de una aplicación web integral destinada a la digitalización de los procesos de limpieza y desinfección en entornos hospitalarios.

Se trata de una **Single Page Application (SPA)** desarrollada con tecnologías web modernas, que permite la gestión en tiempo real de las tareas de higiene, garantizando la trazabilidad de las acciones realizadas por el personal de limpieza y ofreciendo a los supervisores herramientas de control y auditoría.

El sistema sustituye los tradicionales partes de trabajo en papel por una plataforma digital centralizada, accesible desde dispositivos móviles (para los operarios) y equipos de escritorio (para la administración).

1.2 JUSTIFICACIÓN Y PROBLEMÁTICA

Proyecto “Desarrollo de Aplicaciones Web”

Título del Proyecto: SANICLEARS



En la actualidad, la gestión de la higiene en muchos centros sanitarios sigue dependiendo de procesos manuales y registros físicos. Esta metodología presenta varios inconvenientes críticos:

- **Falta de trazabilidad:** Es difícil saber con exactitud quién limpió una zona específica y a qué hora exacta.
- **Retraso en la información:** Las incidencias (como una habitación bloqueada o falta de material) no se comunican en tiempo real.
- **Dificultad de auditoría:** Analizar el rendimiento del personal o el consumo de materiales requiere digitalizar manualmente los datos.

La idea de Saniclears nace de una experiencia personal profunda y reveladora. Durante el año pasado, a raíz del ingreso hospitalario de mi abuela en el Hospital de Mérida, pasé largas horas en el recinto. Allí, tuve la oportunidad de observar de cerca el incansable trabajo del personal de limpieza, entre el cual se encuentra un familiar directo.

Viendo su día a día, me llamó poderosamente la atención un detalle anacrónico en un entorno donde la tecnología médica es puntera: la gestión de la limpieza se seguía llevando a cabo mediante partes de papel y bolígrafo. Los operarios debían firmar hojas colgadas en las puertas, anotar incidencias manualmente y buscar físicamente a sus supervisores para reportar problemas de material o mantenimiento. Esta falta de digitalización no solo ralentizaba el trabajo, sino que propiciaba la pérdida de información, dificultaba la trazabilidad real de la desinfección y limitaba la capacidad de reacción ante urgencias.

De esta necesidad palpable y cotidiana surgió Saniclears: una plataforma web integral diseñada para digitalizar, optimizar y monitorizar en tiempo real la higiene hospitalaria. El objetivo es dotar al personal de limpieza de herramientas modernas que dignifiquen su trabajo, y a la administración de un control exhaustivo que garantice la máxima seguridad y salubridad para los pacientes.

1.3 ÁMBITO DEL PROYECTO

El alcance del sistema abarca los siguientes módulos funcionales:

- **Gestión de Usuarios y Roles:** Control de acceso seguro mediante autenticación y roles diferenciados.
- **Gestión de Espacios (Zonas):** Catálogo digital de todas las áreas del hospital categorizadas por nivel de riesgo biológico.
- **Planificación de Tareas:** Asignación dinámica de tareas de limpieza a los operarios.
- **Control de Incidencias:** Sistema de reporte de problemas en tiempo real.
- **Notificaciones Internas:** Sistema de mensajería segmentada por turnos.
- **Panel Estadístico Multicéntrico:** Métricas globales y predictivas para el Superadmin.

Proyecto “Desarrollo de Aplicaciones Web”

Título del Proyecto: SANICLEARS



Perfil de Usuario	Rol Técnico (DB)	Descripción y Responsabilidades	Permisos en el Sistema
Megasupervisor	superadmin	Gestión multicéntrica de todas las entidades.	Acceso total al panel global y métricas multicéntricas.
Supervisor	admin	Encargado de la gestión integral de la plataforma: administración de recursos humanos y materiales, supervisión del estado de higiene del hospital.	CRUD completo. Gestión de usuarios, zonas y turnos. Auditoría y resolución de incidencias.
Operario	operario (ID: 1)	Personal de limpieza encargado de la ejecución de tareas. Interfaz simplificada para uso rápido en móvil o tablet.	Solo visualiza sus tareas asignadas. Puede marcar tareas como completadas y crear reportes de incidencias.

1.4 DIAGRAMA DE CONTEXTO

Para comprender el funcionamiento global de **Saniclears**, se presenta el siguiente esquema de interacción entre los usuarios y el sistema.

El flujo de información comienza cuando el usuario (MegaSupervisor, Supervisor u Operario) accede a la aplicación a través de un navegador web. La aplicación cliente (Frontend) se comunica de forma segura mediante HTTPS con los servicios en la nube de Supabase, donde reside la lógica de negocio y la base de datos.

El sistema es ****Multi-Tenant****: la tabla `entidades` (Hospitales) actúa como eje central para separar los datos de distintos centros sanitarios mediante `entidad\_id`.

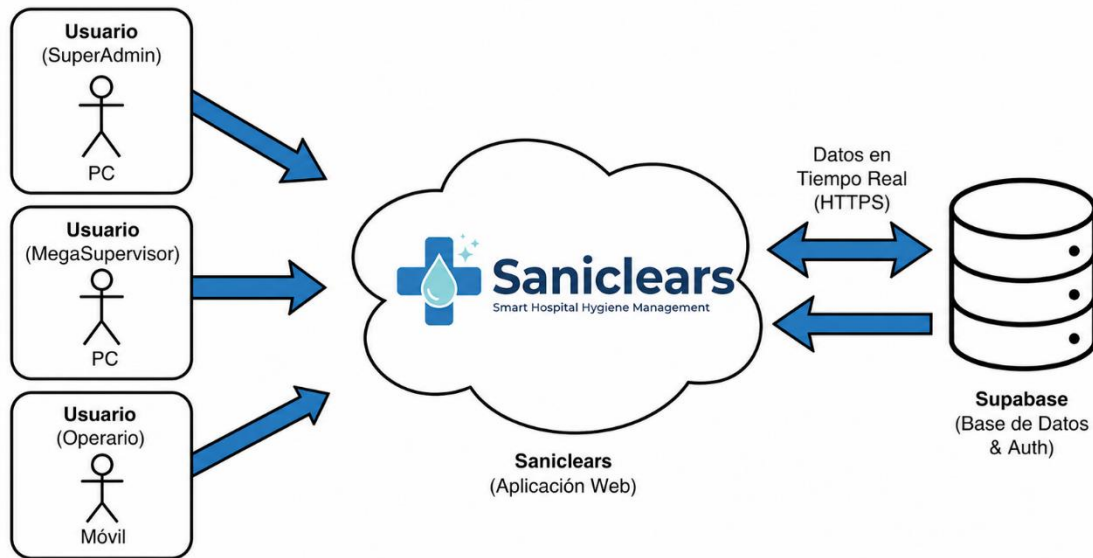


Figura 1.1. Diagrama de Contexto del Sistema Saniclears.

2. ARQUITECTURA DE LA APLICACIÓN

El proyecto **Saniclears** se ha diseñado siguiendo un modelo de arquitectura moderna, desacoplada y basada en la nube. A diferencia de las arquitecturas monolíticas tradicionales (donde interfaz y lógica residen en el mismo servidor), este sistema implementa una arquitectura **Serverless** y **SPA (Single Page Application)**.

Esta separación permite un desarrollo más ágil, mayor escalabilidad y una experiencia de usuario más fluida. El sistema se divide claramente en dos bloques que se comunican mediante peticiones asíncronas seguras (HTTPS).

2.1 FRONTED

2.1.1 TECNOLOGÍAS USADAS

Para la construcción de la interfaz se ha seleccionado un stack tecnológico basado en el ecosistema de JavaScript moderno:

- **React (v19)+ Typescript:** Librería principal utilizada para construir la interfaz de usuario basada en componentes reutilizables. Permite una gestión eficiente del estado de la aplicación.
- **Vite 7:** Herramienta de construcción (bundler) de próxima generación. Se ha elegido por su velocidad de arranque y su Hot Module Replacement (HMR) instantáneo, lo que agiliza el desarrollo frente a opciones clásicas como Webpack.

Proyecto “Desarrollo de Aplicaciones Web”

Título del Proyecto: SANICLEARS



- **Tailwind CSS 4:** Framework de diseño "utility-first". Permite maquetar la aplicación de forma rápida y responsive directamente desde el HTML (JSX), garantizando que la app se adapte a móviles y ordenadores.
- **React Router DOM:** Librería estándar para gestionar el enrutamiento dinámico en una SPA, permitiendo navegar entre las vistas (Login, Dashboard, Tareas) sin recargar la página completa.
- **Lucide React:** Colección de iconos vectoriales ligeros y consistentes para la interfaz gráfica.
- **Zustand:** para el estado global (como la sesión del usuario) y React Context API.
- **Gráficas y Animaciones:** Recharts para visualización de datos estadísticos y GSAP para animaciones fluidas de la interfaz.
- **Supabase Client (@supabase/supabase-js):** Librería oficial que permite conectar el Frontend con el Backend, gestionando la autenticación y las consultas a la base de datos de forma segura.

2.1.2 ENTORNOS DE DESARROLLO

El entorno de trabajo configurado para el desarrollo del cliente incluye:

- **Node.js & NPM:** Entorno de ejecución y gestor de paquetes necesario para instalar y ejecutar las librerías de React.
- **Visual Studio Code (VS Code):** IDE principal utilizado, potenciado con extensiones como ESLint (para calidad de código), Prettier (formato) y Tailwind CSS IntelliSense.
- **Git & GitHub:** Sistema de control de versiones para gestionar el código fuente y las ramas de desarrollo (main, dev).
- **Navegadores y DevTools:** Se han utilizado Google Chrome y sus herramientas de desarrollo para la depuración y las pruebas de diseño responsive en diferentes resoluciones.
- **Supabase Storage:** Buckets de almacenamiento en la nube para gestionar archivos estáticos como fotografías adjuntas en las incidencias.
- **Realtime (WebSockets):** Suscripciones a cambios en la base de datos para actualizar los paneles de control en vivo.

2.2 BACKEND

Para la lógica del servidor y la persistencia de datos, Saniclear prescinde de un servidor backend tradicional (como Node.js Express o PHP) y adopta un modelo **BaaS (Backend-as-a-Service)**. Esto delega la infraestructura en la plataforma **Supabase**, lo que garantiza alta disponibilidad y seguridad desde el primer día.

2.2.1 TECNOLOGÍAS USADAS

La infraestructura del lado del servidor se compone de:

- **Supabase:** Plataforma de código abierto que proporciona toda la infraestructura de backend.
- **PostgreSQL:** El motor de base de datos relacional sobre el que se sustenta el sistema. Es conocido por su robustez y fiabilidad.
- **Supabase Auth (GoTrue):** Servicio de autenticación integrado. Gestiona el registro de usuarios, el inicio de sesión seguro y la emisión de tokens JWT (JSON Web Tokens) para mantener la sesión.
- **Row Level Security (RLS):** Sistema de políticas de seguridad nativo de PostgreSQL. Es fundamental en esta arquitectura, ya que define qué filas puede leer o editar cada usuario según su rol (Superadmin, Admin u Operario).
- **PL/pgSQL (Triggers):** Lenguaje procedimental utilizado para programar la lógica de negocio directamente en la base de datos. Se utiliza, por ejemplo, para asignar roles automáticamente cuando un usuario se registra.

2.2.2 ENTORNO DE DESARROLLO

La gestión y programación del backend se realiza a través de:

- **Supabase Dashboard:** Panel de administración web desde el cual se gestionan las tablas de la base de datos, los usuarios registrados y las políticas de seguridad.
- **SQL Editor:** Interfaz integrada en Supabase donde se escriben y ejecutan los scripts SQL para la creación de tablas, funciones y triggers.
- **Table Editor:** Herramienta visual del dashboard utilizada para la inspección rápida de datos y verificación de registros durante las pruebas.



3. DOCUMENTACIÓN TÉCNICA

En este apartado se detalla la estructura interna de los datos y la lógica de negocio implementada en el backend para garantizar el funcionamiento de SANICLEARS.

3.1 ANÁLISIS

El sistema se sustenta en una base de datos relacional PostgreSQL, alojada en la nube mediante Supabase. El diseño se ha normalizado para evitar redundancias y asegurar la integridad referencial.
A continuación, se describen las entidades principales del esquema:

3.1.1 REQUISITOS FUNCIONALES

A continuación, se detallan los requisitos funcionales del sistema, organizados por código identificador:

ID	Nombre	Descripción
RF01	Gestión de Roles	El sistema distingue entre Superadmin (gestión multicéntrica), Administrador (gestión local del hospital) y Operario (ejecución de tareas).
RF02	Gestión de Usuarios y Zonas	El administrador puede realizar un CRUD completo del personal y de las áreas del hospital.
RF03	Asignación de Tareas	Creación, asignación y seguimiento de tareas de limpieza con niveles de prioridad.
RF04	Reporte de Incidencias	Los operarios pueden reportar problemas (roturas, falta de material) adjuntando descripciones y fotografías.
RF05	Notificaciones Internas	Sistema de mensajería para avisos urgentes o cambios de protocolo segmentado por turnos.
RF06	Panel Estadístico	El Superadmin visualiza métricas de carga operativa, rendimiento y alertas predictivas.

3.1.2 REQUISITOS NO FUNCIONALES

Proyecto “Desarrollo de Aplicaciones Web”

Título del Proyecto: SANICLEARS



ID	Nombre	Descripción
RNF01	Usabilidad y Responsive	La interfaz del operario debe ser 100% usable en teléfonos móviles, con botones grandes y flujos intuitivos.
RNF02	Rendimiento	Carga inicial inferior a 2 segundos; navegación SPA sin recargas de página completas.
RNF03	Seguridad	Las contraseñas deben estar encriptadas. Todo endpoint y consulta debe validar el token JWT y el rol del emisor.
RNF04	Disponibilidad	El sistema debe estar alojado en la nube con alta disponibilidad (99.9% uptime).

3.2 DESARROLLOS

3.2.1) DISEÑO DE LA BBDD

La base de datos aprovecha el ecosistema relacional de PostgreSQL. Se ha diseñado una arquitectura Multi-Tenant, donde la tabla entidades (Hospitales) actúa como eje central para separar los datos de distintos centros sanitarios.

Se hace uso de Triggers en PostgreSQL para automatizar procesos; por ejemplo, cuando un usuario se registra en auth.users de Supabase, un trigger copia automáticamente sus datos públicos a nuestra tabla public.usuarios.

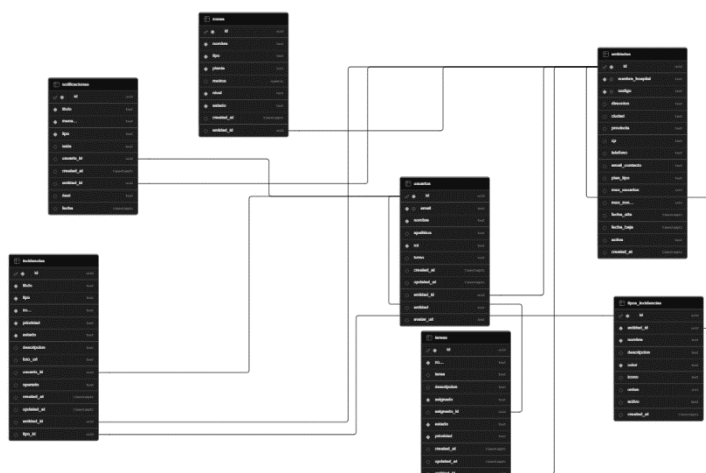
Proyecto “Desarrollo de Aplicaciones Web”

Título del Proyecto: SANICLEARS



JUNTA DE EXTREMADURA

Consejería de Educación y Empleo



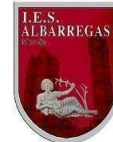
3.2.2) MODELO ENTIDAD/RELACIÓN DE LA BBDD

Las entidades principales del sistema y sus cardinalidades son las siguientes:

Entidad	Cardinalidad	Relación
Entidad	(1) — (N)	Un hospital tiene muchos trabajadores.
Entidad	(1) — (N)	Un hospital se divide en muchas zonas.
Zona	(1) — (N)	En una zona se pueden realizar múltiples tareas.
Usuario	(1) — (N)	Un operario puede tener asignadas múltiples tareas.
Usuario	(1) — (N)	Un operario puede reportar muchas incidencias.
Entidad	(1) — (N)	Un hospital emite múltiples notificaciones a su plantilla.

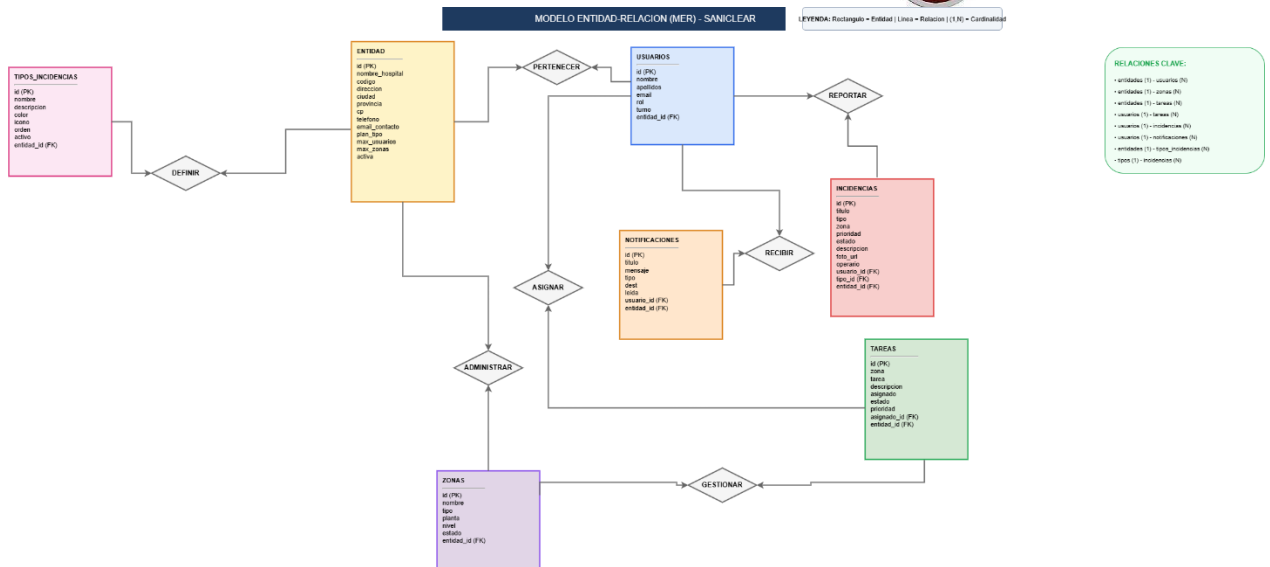
Proyecto “Desarrollo de Aplicaciones Web”

Título del Proyecto: SANICLEARS



JUNTA DE EXTREMADURA

Consejería de Educación y Empleo



3.2.3 MODELO RELACIONAL DE LA BBDD

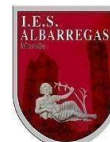
A continuación, se muestra el modelo relacional de la base de datos con sus tablas, campos y claves:

entidades

Campo	Tipo de Dato	Clave	Null
id	UUID	PK	NO
nombre_hospital	TEXT	-	NO
codigo	TEXT	-	NO
direccion	TEXT	-	Si
Ciudad	TEXT	-	Si
Provincia	TEXT	-	Si

Proyecto “Desarrollo de Aplicaciones Web”

Título del Proyecto: SANICLEARS



JUNTA DE EXTREMADURA

Consejería de Educación y Empleo

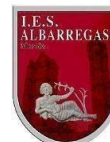
Cp	TEXT	-	Si
Teléfono	TEXT	-	Si
Email_contacto	TEXT	-	Si
Plan_tipo	TEXT	-	Si
Max_usuarios	INTEGER	-	Si
Max_zonas	INTEGER	-	Si
Fecha_alta	TIMESTAMPZ	-	Si
Fecha_baja	TIMESTAMPZ	-	Si
Activa	BOOLEAN	-	Si
Created_at	TIMESTAMPZ	-	Si

usuarios

Campo	Tipo de Dato	Clave	Null
id	UUID	PK	NO
Entidad_id	UUID	FK	SI
email	TEXT	-	NO
nombre	TEXT	-	Si
apellidos	TEXT	-	Si

Proyecto “Desarrollo de Aplicaciones Web”

Título del Proyecto: SANICLEARS



JUNTA DE EXTREMADURA

Consejería de Educación y Empleo

rol	TEXT	-	NO
turno	TEXT	-	Si
entidad	TEXT	-	Si
Avatar_url	TEXT	-	Si
Created_at	TIMESTAMPTZ	-	Si
updated_at	TIMESTAMPTZ	-	Si

zonas

Campo	Tipo de Dato	Clave	Null
id	UUID	PK	NO
Entidad_id	UUID	FK	SI
nombre	TEXT	-	NO
tipo	TEXT	-	NO
planta	INTEGER	-	NO
metros	NUMBER	-	Si
nivel	TEXT	-	NO
estado	TEXT	-	NO
Created_at	TIMESTAMPTZ	-	Si

Proyecto “Desarrollo de Aplicaciones Web”

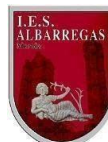
Título del Proyecto: SANICLEARS



tareas			
Campo	Tipo de Dato	Clave	Null
id	UUID	PK	NO
Entidad_id	UUID	FK	SI
Asignado_id	UUID	FK	SI
zona	TEXT	-	NO
tarea	TEXT	-	Si
descripcion	TEXT	-	Si
Asignado	TEXT	-	NO
Estado	TEXT	-	NO
Prioridad	TEXT	-	NO
Created_at	TIMESTAMPTZ	-	Si
Updated_at	TIMESTAMPTZ	-	Si
tipos_incidentes			
Campo	Tipo de Dato	Clave	Null

Proyecto “Desarrollo de Aplicaciones Web”

Título del Proyecto: SANICLEARS



JUNTA DE EXTREMADURA

Consejería de Educación y Empleo

id	UUID	PK	NO
Entidad_id	UUID	FK	NO
Nombre	TEXT	-	NO
Descripcion	TEXT	-	Si
Color	TEXT	-	NO
Icono	TEXT	-	Si
Orden	INTERGER	-	Si
activo	BOOLEAN	-	Si
Created_at	TIMESTAMPZ	-	Si

incidencias

Campo	Tipo de Dato	Clave	Null
id	UUID	PK	NO
Entidad_id	UUID	FK	SI
Tipo_id	UUID	FK	SI
Usuario_id	UUID	FK	Si
Titulo	TEXT	-	NO

Proyecto “Desarrollo de Aplicaciones Web”

Título del Proyecto: SANICLEARS



JUNTA DE EXTREMADURA

Consejería de Educación y Empleo

tipo	TEXT	-	NO
zona	TEXT	-	NO
prioridad	TEXT	-	NO
estado	TEXT	-	NO
descripcion	TEXT	-	Si
Foto_url	TEXT	-	Si
operario	TEXT	-	Si
Created_at	TIMESTAMP TZ	-	Si
Updated_at	TIMESTAMP TZ	-	Si
notificaciones			
Campo	Tipo de Dato	Clave	Null
id	UUID	PK	NO
Entidad_id	UUID	FK	SI
Usuario_id	UUID	FK	SI
titulo	TEXT	-	NO
mensaje	TEXT	-	NO

Proyecto “Desarrollo de Aplicaciones Web”

Título del Proyecto: SANICLEARS



tipo	TEXT	-	NO
leida	BOOLEAN	-	Si
Dest (destinatario)	TEXT	-	Si
fecha	TIMESTAMPZ	-	Si
Created_at	TIMESTAMPZ	-	Si



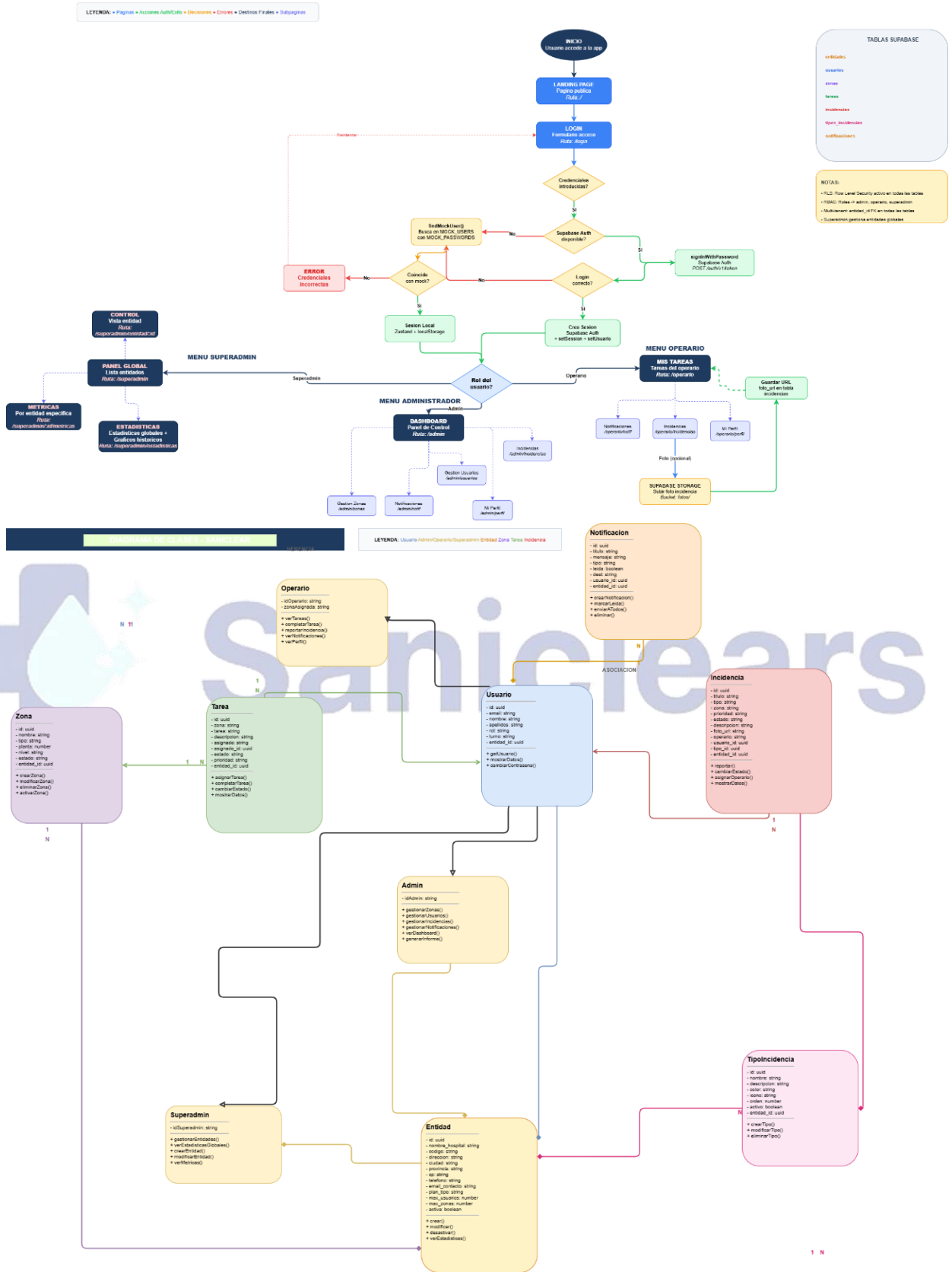
Título del Proyecto: SANICLEARS



JUNTA DE EXTREMADURA

Consejería de Educación y Empleo

DIAGRAMA DE FLUJO - SANICLEAR



3.2.4 DIAGRAMA DE CASOS DE USO

A continuación, se describen los casos de uso por actor del sistema:

Actor: Operario

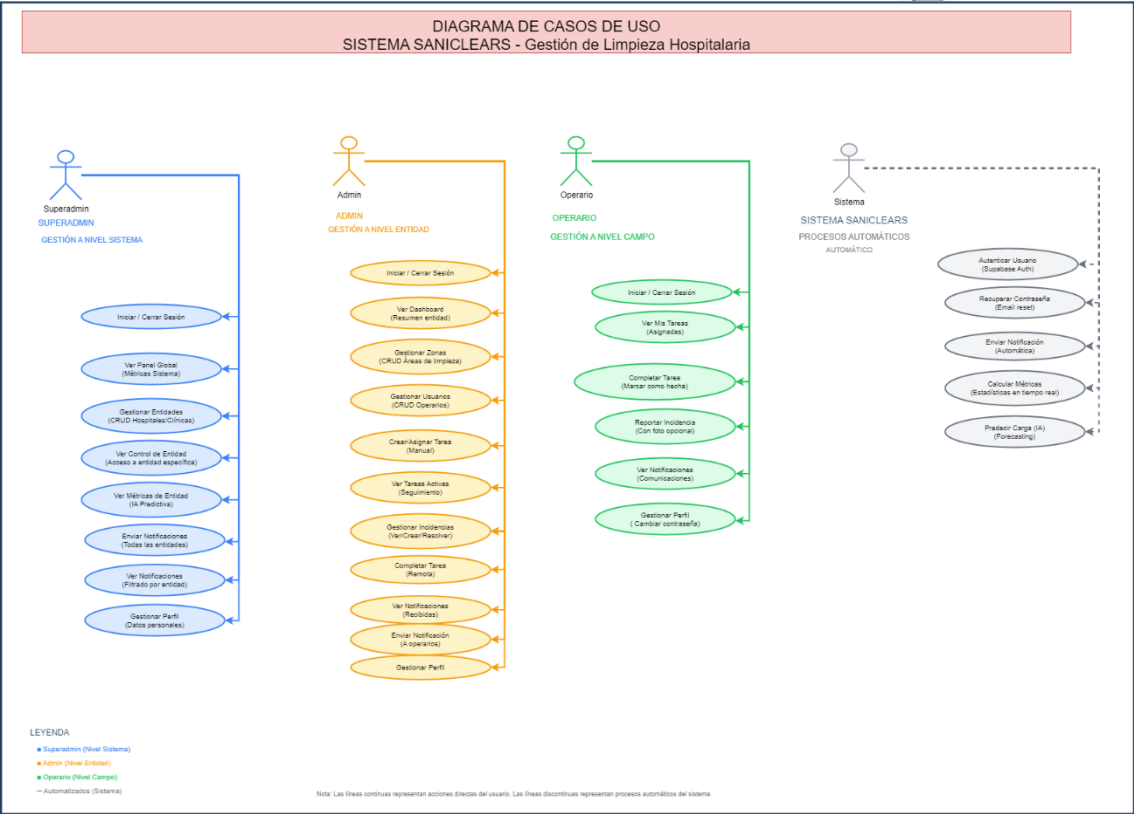
- Iniciar Sesión.
- Ver lista de tareas diarias asignadas.
- Marcar tarea como 'En Curso' o 'Completada'.
- Reportar una nueva incidencia (con foto opcional).
- Leer notificaciones recibidas.

Actor: Administrador

- Acceder al Dashboard Local (estadísticas de su hospital).
- Crear, modificar y eliminar Zonas y Usuarios Operarios.
- Asignar Tareas a operarios.
- Cambiar estado de Incidencias (a 'En Revisión' o 'Resuelta') y añadir notas.
- Emitir Notificaciones a los turnos.

Actor: Superadmin

- Visualizar Panel Global con carga operativa multicéntrica.
- Filtrar métricas e histórico por Entidad (Hospital).
- Visualizar predicciones operativas.



3.3 PRUEBAS REALIZADAS

Para garantizar la fiabilidad del sistema antes de su puesta en producción, se ha ejecutado un plan de pruebas exhaustivo que abarca desde la seguridad de los datos hasta la adaptabilidad de la interfaz.

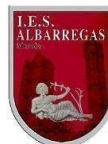
3.3.1 PRUEBAS FUNCIONALES

Se ha verificado el correcto funcionamiento de los casos de uso principales mediante pruebas de caja negra.

ID Prueba	Caso de Uso	Acción Realizada	Resultado Esperado	Resultado Obtenido	Estado
PF-01	Login Correcto	Usuario introduce credenciales válidas.	Redirección al Dashboard correspondiente según su rol.	Redirección correcta y token JWT generado.	OK

Proyecto “Desarrollo de Aplicaciones Web”

Título del Proyecto: SANICLEARS



JUNTA DE EXTREMADURA

Consejería de Educación y Empleo

PF-02	Login Erróneo	Usuario introduce contraseña incorrecta.	Mensaje de error visual en pantalla.	Mensaje "Credenciales inválidas" mostrado.	OK
PF-03	Crear Tarea	Administrador asigna tarea a Operario X.	La tarea aparece en la BD y en la vista "Mis Tareas" del Operario X.	Tarea creada y visible en tiempo real.	OK
PF-04	Completar Tarea	Operario pulsa el check de tarea terminada.	El estado cambia a "completada" y desaparece de tareas activas.	Estado actualizado correctamente en BD.	OK
PF-05	Reportar Incidencia	Operario envía incidencia marcada como "Urgente".	Se guarda en BD con prioridad 'crítica' y estado 'abierta'.	Incidencia reportada y visible para Admin.	OK
PF-06	Resolver Incidencia	Administrador añade nota y marca "Resuelta".	Incidencia se actualiza en la tabla y muestra el comentario.	Estado y nota guardados con éxito.	OK
PF-07	Crear Notificación	Administrador envía aviso al "Turno de mañana".	Aparece notificación no leída solo a operarios de ese turno.	Notificación segmentada recibida.	OK
PF-08	Acceso Superadmin	Superadmin accede al Panel Global.	Se muestran métricas consolidadas (sumatorio) de todos los hospitales.	Métricas multicéntricas cargadas con éxito.	OK
PF-09	Filtro de Entidad	Superadmin selecciona un hospital específico en el panel.	Contadores, gráficas y alertas se recalculan solo para ese hospital.	Datos filtrados correctamente.	OK
PF-10	Filtro Histórico	Superadmin cambia el selector de mes en la gráfica de carga operativa.	La gráfica refleja exclusivamente tareas e incidencias del mes elegido.	Gráfica y leyendas actualizadas.	OK
PF-11	Aviso Global	Superadmin envía notificación a "Todas" las entidades.	Operarios de todos los hospitales del sistema reciben el aviso.	Envío multicéntrico masivo exitoso.	OK
PF-12	Seguridad RLS	Operario intenta acceder forzosamente a la URL /admin.	El sistema bloquea el acceso y redirige de vuelta a /operario.	Acceso denegado (Protección de rutas activa).	OK

3.3.2 PRUEBAS DE SEGURIDAD (RLS)

Dada la sensibilidad de los datos hospitalarios, se han validado las Políticas de Seguridad a Nivel de Fila (RLS) en Supabase.

- **Prueba de Aislamiento:** Se intentó acceder a la tabla usuarios desde una cuenta de Operario mediante la consola del navegador.
 - *Resultado:* Supabase denegó la lectura de datos de otros usuarios, devolviendo un array vacío o error 403, confirmando que la política SELECT funciona correctamente.
- **Prueba de Integridad:** Se intentó eliminar una Zona con una cuenta de Operario.
 - *Resultado:* La base de datos rechazó la operación DELETE, ya que solo el rol admin tiene permiso de escritura en esa tabla.

3.3.3 PRUEBAS DE INTERFAZ Y RESPONSIVIDAD

Se ha verificado la visualización en distintos dispositivos utilizando las herramientas de desarrollo de Chrome (Device Mode):

- **Escritorio (1920x1080):** El Dashboard muestra los gráficos y tablas expandidos.
- **Tablet (iPad Air):** El menú lateral se adapta o se convierte en menú hamburguesa.
- **Móvil (iPhone SE / Pixel 5):** Las tablas complejas se transforman en tarjetas (Cards) para facilitar la lectura vertical, y los botones de acción aumentan de tamaño para ser pulsables con el dedo.

3.4 IMPLEMENTACIÓN DEL AGENTE IA (SANICLEARBOT)

El asistente inteligente integrado en la aplicación representa un núcleo de soporte avanzado tanto para operarios como para administradores. Utiliza la tecnología de **Google Generative AI** a través del modelo **Gemini 1.5 Flash**, seleccionado por su baja latencia y alta capacidad de procesamiento de contexto.

3.4.1 ARQUITECTURA DEL AGENTE

La implementación del bot se ha estructurado de forma modular para garantizar la escalabilidad y el mantenimiento del código:

- **Capa de Servicio (src/services/gemini.ts):** Módulo encargado de la comunicación con la API de Google AI Studio. Gestiona la autenticación mediante claves de entorno y la configuración del modelo (temperatura, límites de respuesta, etc.).
- **Interfaz de Usuario (src/components/common/AIAssistant/):** Componente React que despliega un chat flotante interactivo. Incluye gestión de estados para el historial de mensajes y animaciones de carga (*feedback* visual).

Proyecto “Desarrollo de Aplicaciones Web”

Título del Proyecto: SANICLEARS



- **Base de Conocimiento Dinámica:** Se utiliza un archivo de documentación en la carpeta documentación que actúa como la "fuente de la verdad". Este archivo se importa como texto plano y se inyecta en el *System Prompt* del modelo.

3.4.2 ESPECIFICACIONES TÉCNICAS Y LÓGICA DE INYECCIÓN

El factor diferenciador de **SaniclearsBot** es el uso de **Grounding** (anclaje a la realidad). En lugar de permitir que la IA responda de forma generalista, se le restringe a la documentación específica del proyecto.

3.4.3 FUNCIONALIDADES DEL ASISTENTE

El asistente ofrece una serie de características avanzadas que mejoran la experiencia de usuario (UX):

1. **Respuestas Basadas en Datos Reales:** Gracias a la inyección de contexto, el bot conoce la arquitectura de la base de datos, los roles de usuario y las funcionalidades de cada módulo.
2. **Memoria de Sesión:** Mantiene el hilo de la conversación durante la sesión activa, permitiendo preguntas de seguimiento (ej: "*¿Cómo reporto una incidencia?*" seguido de "*¿Y quién la recibe?*").
3. **Formato Enriquecido:** Utiliza react-markdown para presentar la información de manera legible mediante listas, tablas comparativas y bloques de código resaltados.
4. **Acceso Global:** Accesible mediante un botón flotante con diseño corporativo presente en todas las vistas de la aplicación, facilitando la ayuda contextual inmediata.

3.5.4. CASOS DE USO RECOMENDADOS

- **Guía para el Tribunal:** Explicación de la arquitectura técnica, el stack tecnológico y la lógica de los roles Superadmin.
- **Soporte al Operario:** Resolución de dudas sobre cómo marcar tareas o qué hacer ante una incidencia crítica.
- **Credenciales de Demo:** El bot está programado para facilitar las cuentas de prueba al tribunal, agilizando la fase de evaluación.

4. PROCESO DE DESPLIEGUE

El despliegue de SANICLEARS se realiza siguiendo una arquitectura desacoplada: el frontend (React + Vite) se sirve desde una CDN global (Vercel) mientras que el backend (Supabase BaaS) opera de forma independiente en la nube. No existe un servidor backend tradicional que desplegar; la lógica de negocio reside en la base de datos PostgreSQL y se accede a través de la API REST de Supabase mediante API keys configuradas en variables de entorno.

Esta separación permite:

- Escalabilidad horizontal automática del frontend (CDN edge)
- Mantenimiento independiente de cada capa (frontend/backend)
- Cero tiempo de inactividad durante actualizaciones (immutable deployments)
- Reducción de costes operativos (sin gestión de servidores VM)

La sección 4 documenta, paso a paso, la puesta en producción del sistema:

adquisición del dominio y correo corporativo en Cloudflare, revisiones previas, creación del proyecto en Supabase, ejecución del esquema SQL, obtención de credenciales, configuración de variables de entorno y despliegue automático del frontend en Vercel mediante CI/CD.

4.1 ADQUISICIÓN DE DOMINIO Y CORREO CORPORATIVO EN CLOUDFLARE

Para proyectar una imagen profesional y centralizar las comunicaciones oficiales, se adquiere un dominio personalizado y se configura un buzón de correo corporativo a través de Cloudflare, que actúa como registrador DNS y proveedor de email hosting con filtrado anti-spam y protección avanzada.

Proyecto “Desarrollo de Aplicaciones Web”

Título del Proyecto: SANICLEARS



JUNTA DE EXTREMADURA

Consejería de Educación y Empleo

PASOS:

1. Registrar cuenta en Vercel

- Acceder a cloudflare.com y crear cuenta gratuita.

2. Comprar dominio personalizado

- Ir a "Registrar Dominios" → Buscar "saniclears.com".
- Completar la compra.

3. Contratar plan de correo en Cloudflare

- En el panel del dominio, ir a "Email Routing" → "Email Addresses".
- Añadir buzón: superadmin@saniclear.com
- Configurar reenvío a cuenta personal (Gmail/Outlook) o usar Cloudflare Email Workers para acceso IMAP/SMTP completo.
- Verificar dominio añadiendo registros MX y TXT en DNS.

4. Verificar propiedad del dominio

- Cloudflare proporciona un registro TXT de verificación.
- Añadirlo en la zona DNS del dominio. Esperar propagación (1–2h).

5. Configurar DNS para apuntar a Vercel

- Una vez verificado, añadir registro CNAME:
Nombre: @ (o www) → Valor: cname.vercel-dns.com
- Asegurar que los nameservers del dominio apuntan a Cloudflare (ns1.cloudflare.com, ns2.cloudflare.com).

Resultado:

- Correo oficial corporativo: superadmin@saniclear.com
- Sitio web accesible en: <https://saniclear.com>
- Frontend desplegado en Vercel con dominio custom configurado.

4.2 DESPLIEGUE DEL BACKEND

La estructura de la base de datos no se sube como archivos: se define ejecutando el script SQL de esquema directamente en el entorno de Supabase.

Proyecto “Desarrollo de Aplicaciones Web”

Título del Proyecto: SANICLEARS



JUNTA DE EXTREMADURA

Consejería de Educación y Empleo

1. Crear el proyecto en Supabase:
 - Acceder a supabase.com, crear una nueva organización y un proyecto.
 - Elegir región europea (eu-central-1) para cumplir con el RGPD.
2. Ejecutar el script de esquema:
 - Desde SQL Editor → New Query, pegar y ejecutar el archivo schema.sql del repositorio.
 - El script crea las tablas, tipos ENUM, políticas RLS y triggers.
3. Obtener las credenciales de API:
 - En Project Settings → API, copiar:
 - La URL del proyecto
 - La clave anónima (anon key)
4. Configurar variables de entorno locales:
 - Crear el archivo .env.local en la raíz del proyecto React:

URL del proyecto Supabase

obtenida en Project Settings > API

```
VITE_SUPABASE_URL="https://zwmfzqdamdibjermgnyo.supabase.co"
```

```
# Clave pública anónima — NO es secreta, pero NO debe exponerse en  
# repositorios públicos
```

```
VITE_SUPABASE_ANON_KEY=eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJXXXXXX  
X
```

SEGURIDAD: El archivo **.env.local** debe estar incluido en **.gitignore** para evitar exponer las claves en el repositorio público. Vercel inyectará estas variables a través de su panel de configuración en producción.

4.3 DESPLIEGUE DEL FRONTEND

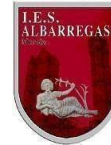
El flujo CI/CD automatizado elimina cualquier paso manual de publicación. Vercel se encarga de compilar, optimizar y distribuir el frontend en su CDN global.

1. Conectar repositorio GitHub con Vercel

- Iniciar sesión en vercel.com y pulsar "New Project".

Proyecto “Desarrollo de Aplicaciones Web”

Título del Proyecto: SANICLEARS



- Importar el repositorio desde GitHub (autorizar acceso).
- Vercel detecta automáticamente que es un proyecto Vite + React.

2. Configurar variables de entorno en Vercel:

- En la configuración del proyecto, ir a "Environment Variables".
- Añadir:
 - **VITE_SUPABASE_URL** =
`https://zwmfzqdamdibjermgnyo.supabase.co`
 - **VITE_SUPABASE_ANON_KEY** =
`eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.XXXXXXXXXX`
- Marcar "Environment" como Production y Preview.

3. Despliegue automático:

- Cada push a la rama main desencadena un build automático.
- Vercel ejecuta: `npm install` → `npm run build`
- Los archivos de `/dist` se publican en una URL temporal tipo:
`https://saniclears-frontend.vercel.app`

4. Añadir dominio personalizado (saniclear.com):

- En proyecto Vercel → Settings → Domains → Add Domain.
- Introducir: `saniclear.com` (y `www.saniclear.com` si aplica).
- Vercel genera registros DNS que debes copiar (CNAME o A records).

5. Finalizar configuración DNS en Cloudflare:

- En el panel de Cloudflare, ir a "DNS" del dominio `saniclear.com`.
- Crear/Añadir los registros que Vercel te haya proporcionado:

TIPO	NOMBRE	VALORES
CNAME	www	cname.vercel-dns.com
A	@ saniclears.com	216.198.79.1

- Asegurar que el proxy de Cloudflare esté activo (naranja) para habilitar CDN, WAF y compresión SSL automática.

6. Forzar HTTPS (SSL/TLS):

- En Cloudflare → SSL/TLS → "Full (strict)" para conexión cifrada.
- Activar "Always Use HTTPS" para redirigir HTTP → HTTPS.
- Activar "Automatic HTTPS Rewrites" para evitar mixed content.

7. Verificar despliegue:

Proyecto “Desarrollo de Aplicaciones Web”

Título del Proyecto: SANICLEARS



JUNTA DE EXTREMADURA

Consejería de Educación y Empleo

- Acceder a <https://saniclears.com>
- Comprobar que la aplicación carga correctamente y las llamadas a
- Supabase funcionan (login, datos en tiempo real).
- Probar ruta protegida: <https://saniclears.com/admin>

Resultado:

- Frontend en producción: <https://saniclears.com>
- Backend (Supabase): Usa el dominio proporcionado por Supabase
- Correo corporativo: superadmin@saniclear.com funcionando en Cloudflare
- CI/CD activo: cada push a main actualiza el sitio automáticamente

4.4 REQUISITOS DEL SISTEMA

Herramienta	Versión Mínima	Uso
Node.js	≥ 18.0.0	Entorno de ejecución. Necesario para NPM y Vite.
NPM	≥ 9.0.0	Gestor de paquetes para instalar dependencias.
Git	≥ 2.40	Control de versiones y conexión con GitHub/Vercel.
Cuenta Supabase	Free Tier	Proyecto PostgreSQL + Auth en la nube.
Cuenta Vercel	Hobby gratis	Hosting del frontend con CDN global.
Cuenta en Cloudflare	free	Correo recibir peticiones y mensajes.

5. MANUAL DE OPERACIÓN

5.1 ROLES Y PERMISOS

Rol	FUNCIÓN PRINCIPAL	HERRAMIENTAS CLAVE
SUPERADMIN	Control global de negocio multicéntrico	Panel global, control por entidades, estadísticas con predicción IA, notificaciones masivas.
ADMIN	Gestión local del hospital	Control de zonas, plantilla, asignación de tareas, gestión de incidencias.
OPERARIO	Ejecución de tareas	Lista de tareas diarias, reporte de fallos con foto, notificaciones.

5.2 MÓDULO SUPERADMIN — DESCRIPCIÓN DETALLADA

El Superadmin dispone de los siguientes paneles exclusivos orientados al control y escalabilidad del modelo de negocio:

Panel Global (/superadmin):

- Métricas consolidadas de todos los hospitales: usuarios activos, zonas, carga operativa, alertas críticas.
- Filtrado por entidad específica o visión global.
- Filtrado por mes para análisis histórico.
- Gráfica de barras de carga por zona (tareas vs incidencias).
- Panel de alertas y actividad del sistema en tiempo real.

Control de Entidades (/superadmin/entidades):

- Listado de todos los hospitales registrados con búsqueda en tiempo real.
- Crear, editar y eliminar entidades con confirmación de seguridad.
- Acceso directo al panel de control de cada hospital.
- Acceso a métricas individuales de cada entidad.

Proyecto “Desarrollo de Aplicaciones Web”

Título del Proyecto: SANICLEARS



Panel de Control por Entidad (/superadmin/entidades/:id):

- Contadores de usuarios, zonas, tareas activas e incidencias de esa entidad específica.
- CRUD completo de tareas, incidencias, personal y notificaciones de la entidad.
- Capacidad de completar tareas y resolver incidencias de forma remota.
- Vista integral del personal con sus respectivos roles y turnos.

Estadísticas con IA (/superadmin/entidades/:id/estadisticas):

- Gráficas lineales de evolución mensual (incidencias, tareas completadas, carga, productividad).
- Motor predictivo basado en media móvil de 3 periodos y extrapolación lineal.
- Predicción de incidencias para el próximo mes.
- Carga operativa prevista y recomendación de personal óptimo.
- Detección de nivel de riesgo de saturación.
- Identificación de zonas con mayor presión operativa (histórico de tareas e incidencias).

Estadísticas Globales (/superadmin sección estadísticas):

- Evolución histórica sumada de todos los centros gestionados.
- Previsiones de carga e incidencias a nivel global.
- Motor predictivo con media móvil y proyección lineal a gran escala.

5.3 SANICLEARSBOT — ASISTENTE DE IA

El sistema incluye un asistente inteligente integrado, alimentado con **Google AI Studio** y el modelo **Gemini**. Está accesible permanentemente desde el botón flotante (icono de robot) en la esquina inferior derecha de cualquier página de la aplicación.

Capacidades Principales:

- Explicar el uso de cualquier módulo o vista de la aplicación.
- Proporcionar credenciales de prueba para facilitar la defensa del TFG al tribunal.
- Resumir la arquitectura técnica del proyecto.

Proyecto “Desarrollo de Aplicaciones Web”

Título del Proyecto: SANICLEARS



- Detallar el modelo de la base de datos, el sistema de roles (RBAC) y la seguridad (RLS).
- Responder preguntas del tribunal evaluador sobre las decisiones del proyecto.
- Explicar el proceso de despliegue y el stack tecnológico utilizado.
- Describir detalladamente las funcionalidades del Superadmin y su motor predictivo.

Grounding (Anclaje de Contexto): Toda la documentación técnica del TFG se inyecta como contexto en cada conversación de forma invisible para el usuario. Esto garantiza que la IA proporciona respuestas precisas, verídicas y estrictamente ceñidas a la arquitectura del proyecto Saniclears, evitando "alucinaciones".

5.4 FLUJO DE TRABAJO TÍPICO

- 6 El **Administrador** crea una zona (ej. *Quirófano 1*) y asigna una tarea con prioridad alta a un operario.
- 7 El **Operario** recibe la tarea en su dispositivo móvil y la marca como "En Curso".
- 8 Si ocurre un problema (ej. *Falta desinfectante especial*), el **Operario** crea una incidencia adjuntando una fotografía desde la cámara de su dispositivo.
- 9 El **Administrador** recibe el aviso en su Dashboard, añade una nota de resolución y cambia el estado de la incidencia.
- 10 El **Operario** finaliza la tarea marcándola como "Completada".
- 11 El **Superadmin** visualiza el impacto de estas acciones en las métricas consolidadas y las predicciones del motor IA a final de mes.

6. PROPUESTA DE MEJORAS

Este proyecto ha seguido una metodología de desarrollo iterativa, lo que ha permitido detectar cuellos de botella y aplicar soluciones técnicas sobre la marcha, así como planificar funcionalidades avanzadas para una segunda fase.

6.1 MEJORAS APLICADAS DURANTE EL DESARROLLO

Durante la fase de análisis y desarrollo, se realizaron cambios significativos respecto a la propuesta inicial para optimizar el rendimiento y la seguridad de **Saniclear**:

1. **Migración a Arquitectura Serverless (Supabase):**
 - *Situación inicial:* Se planteó el uso de una base de datos MySQL gestionada con un backend tradicional en Node.js.

Proyecto “Desarrollo de Aplicaciones Web”

Título del Proyecto: SANICLEARS



JUNTA DE EXTREMADURA

Consejería de Educación y Empleo

- *Mejora aplicada:* Se migró a **Supabase (PostgreSQL)**.
- *Justificación:* Esta decisión redujo el tiempo de desarrollo del backend en un 40% y permitió implementar **RLS (Row Level Security)**, garantizando que la seguridad de los datos resida en la propia base de datos y no dependa de validaciones propensas a errores en el código del servidor.
- 2. **Adopción de Tailwind CSS "Utility-First":**
 - *Situación inicial:* Uso de hojas de estilo CSS estándar o SASS.
 - *Mejora aplicada:* Implementación del framework **Tailwind CSS**.
 - *Justificación:* Permitted construir una interfaz totalmente **responsive** (adaptable a móviles para los operarios) sin escribir cientos de líneas de CSS personalizado, asegurando una coherencia visual en toda la aplicación.
- 3. **Simplificación del Lenguaje (TypeScript a JavaScript):**
 - *Mejora aplicada:* Se optó por utilizar **JavaScript (JSX)** estándar en lugar de TypeScript estricto para el MVP (Producto Mínimo Viable).
 - *Justificación:* Dado el cronograma ajustado del proyecto final, esto agilizó la escritura de código y la configuración del entorno, manteniendo la robustez gracias a las validaciones de tipos de la base de datos PostgreSQL.

6.2 MEJORAS PROPUESTAS PARA FUTURAS VERSIONES

Para escalar el proyecto hacia un producto comercial completo, se han diseñado las siguientes líneas de evolución técnica:

1. Mapa 3D Interactivo (Gemelo Digital):

- Integración de librerías gráficas como Three.js o React Three Fiber.
- Funcionalidad: Visualización tridimensional de la planta del hospital donde las habitaciones cambien de color en tiempo real (Rojo: Sucio / Verde: Limpio) según el estado de las tareas en la base de datos.

2. Sistema de Fichaje Geolocalizado:

- Uso de la Geolocation API del navegador.
- Funcionalidad: Impedir que el operario fiche la "Entrada de Turno" si sus coordenadas GPS no coinciden con las del recinto hospitalario, garantizando la presencialidad.

3. Notificaciones Push (PWA):

- Conversión de la web en una Progressive Web App (PWA).
- Funcionalidad: Permitir el funcionamiento Offline (sin cobertura en sótanos o quirófanos blindados) y sincronizar los datos cuando vuelva la conexión, además de enviar alertas al móvil del operario cuando se le asigne una tarea urgente.

4. Escaneo QR/NFC:

Permitir a los operarios escanear un código en la puerta de la habitación con su teléfono para hacer Check-in y Check-out de la tarea

Proyecto “Desarrollo de Aplicaciones Web”

Título del Proyecto: SANICLEARS



automáticamente.

5. Módulo de Inteligencia Artificial:

- o Análisis de datos históricos de limpieza.
- o Funcionalidad: El sistema podría predecir qué zonas necesitarán refuerzo de limpieza en épocas de alta ocupación (ej: epidemias de gripe) y sugerir turnos optimizados al Supervisor.

6.3 REQUISITOS TÉCNICOS

6.3.1 BACKEND

- **Redis:** Implementación de una caché intermedia para aligerar la carga de consultas repetitivas a la base de datos de PostgreSQL en paneles con mucho tráfico.
- **Supabase Edge Functions:** Migración de ciertas lógicas pesadas, como el envío de correos electrónicos automáticos a mantenimiento cuando se genere una incidencia crítica.

6.3.2 FRONTED

- **PWA (Progressive Web App):** Conversión de la aplicación web a PWA, permitiendo a los operarios instalarla como aplicación nativa y usarla en modo Offline (guardando tareas completadas en caché local y sincronizándolas al recuperar la cobertura).
- **Testing E2E automatizado:** Inclusión de pruebas automatizadas End-to-End con herramientas como Cypress o Playwright.

7. CREDENCIALES DE ACCESO

Nota para el Tribunal: Estas credenciales se proporcionan para uso exclusivo de la defensa y evaluación del TFG, permitiendo la exploración de todos los roles del sistema.

PERFIL	USUARIO	CONTRASEÑA
Superadmin	superadmin@saniclear.com	SuperAdmin1234!
Luis Ramirez(Operario)	luisramirez@gmail.com	Operario123!

Proyecto “Desarrollo de Aplicaciones Web”

Título del Proyecto: SANICLEARS



JUNTA DE EXTREMADURA

Consejería de Educación y Empleo

Kike Ambrona	kikeambrona@gmail.com	Kikea14!
Ana	ana@gmail.com	Operario123!
Admin(maría)	admin@gmail.com	Admin123!
Laura Gómez	lauragomez@gmail.com	Operario123!
Elena Ruíz	elenaruiz@gmail.com	Admin123!
Carmen Díaz	carmendiaz@gmail.com	Operario123!
Alberto Vera	albertovera@gmail.com	Admin123!

8. BIBLIOGRAFÍA

8.1 WEBS DE REFERENCIA

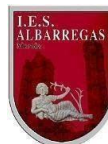
Para el desarrollo de este proyecto se ha consultado la siguiente documentación técnica oficial:

- **React Documentation:** <https://react.dev/> - Referencia para Hooks y componentes.
- **Supabase Docs:** <https://supabase.com/docs> - Guías de autenticación y base de datos.
- **PostgreSQL Documentation:** <https://www.postgresql.org/docs/> - Sintaxis SQL y PL/pgSQL.
- **Tailwind CSS:** <https://tailwindcss.com/> - Referencia de clases de utilidad.
- **Vite Guide:** <https://vitejs.dev/guide/> - Configuración del entorno de construcción.
- **Lucide Icons:** <https://lucide.dev/> - Librería de iconos.
-

8.2 ARTÍCULOS DE REFERENCIA

Proyecto “Desarrollo de Aplicaciones Web”

Título del Proyecto: SANICLEARS



JUNTA DE EXTREMADURA

Consejería de Educación y Empleo

- Role-Based Access Control (RBAC) in Modern Web Applications. Conceptos de seguridad y división de arquitecturas por roles.
- Principios de Diseño Mobile-First para aplicaciones de gestión en terrenos operativos.
- Protocolos de Limpieza y Desinfección en Entornos Sanitarios. Base teórica para estructurar el modelo de datos de Zonas, Prioridades e Incidencias en hospitales.

